

Εργασία 5η - Γράφος - Ατομική Εργασία

Ο Γράφος	1
Το κυρίως πρόγραμμα	3
Έλεγχος του προγράμματος σας	5
Makefile και αρχεία ελέγχου	5
Τρόπος Αποστολής	5
Παράρτημα 1 - Ενδεικτικά γραφήματα	5

Ατομική Εργασία, Καταληκτική ημερομηνία υποβολής: Δευτέρα 17 Ιουνίου

Σε αυτή την εργασία θα υλοποιήσετε την παραμετρική (*templated*) κλάση ενός γράφου. Ο γράφος μπορεί να είναι κατευθυνόμενος ή μη. Για τα μη κατευθυνόμενα γραφήματα μπορείτε να υποθέσετε ότι αυτά θα είναι πάντα συνεκτικά. Η παραπάνω υπόθεση δεν ισχύει για τα κατευθυνόμενα γραφήματα.

Ο Γράφος

Το header file της κλάσης δίνεται παρακάτω:

```
#ifndef _GRAPH_HPP_
#define _GRAPH_HPP_
#include <list>

template<typename T>
struct Edge {
    T from;
    T to;
    int dist;
    Edge(T f, T t, int d): from(f), to(t), dist(d);
    bool operator<(const Edge<T>& e) const; // compare Edges using distance (dist field)
    bool operator>(const Edge<T>& e) const; // compare Edges using distance (dist field)

    template<typename U>
    friend std::ostream& operator<<(std::ostream& out, const Edge<U>& e);
};

template<typename T>
std::ostream& operator<<(std::ostream& out, const Edge<T>& e) {
    out << e.from << " -- " << e.to << " (" << e.dist << ")";
    return out;
}

template <typename T>
class Graph {

public:
    Graph(bool isDirected = true);
    bool addVtx(const T& info);
    bool rmvVtx(const T& info);
```

```

bool addEdg(const T& from, const T& to, int distance);
bool rmvEdg(const T& from, const T& to);
std::list<T> dfs(const T& info) const;
std::list<T> bfs(const T& info) const;
std::list<Edge<T>> mst();

bool print2DotFile(const char *filename) const;
std::list<T> dijkstra(const T& from, const T& to);
std::list<T> bellman_ford(const T& from, const T& to);
};

```

Το γράφημα μπορεί να υλοποιηθεί μέσω πίνακα γειτνιάσεως ή λίστας γειτνιάσεως. Εφόσον υλοποιήσετε το γράφημα μέσω λίστας γειτνιάσεως, συνιστάται η λίστα γειτνιάσεως να υλοποιείται μέσω ενός ~~std::vector~~ ή ~~std::list~~ STL container της επιλογής σας. Κάθε θέση του πίνακα της λίστας αντιστοιχεί σε ένα κόμβο. Σε οποιονδήποτε γράφημα (είτε υλοποιείται μέσω λίστας είτε μέσω πίνακα) δεν είναι απαραίτητο όλες οι θέσεις του πίνακα να είναι κατειλημμένες. Οι συναρτήσεις της κλάσης και ο κατασκευαστής δίνονται παρακάτω:

Μέθοδος	Περιγραφή
<code>Graph(bool isDirected)</code>	Η παράμετρος προσδιορίζει εάν το γράφημα είναι κατευθυνόμενο ή όχι.
<code>bool contains(const T& info);</code>	Επιστρέφει true εάν ο γράφος περιέχει το συγκεκριμένο κόμβο, διαφορετικά false .
<code>bool addVtx(const T& info);</code>	Ενθέτει τον κόμβο στον γράφο, εφόσον ο συγκεκριμένος κόμβος δεν υπάρχει ήδη σε αυτόν. Επιστρέφει true σε περίπτωση επιτυχίας, διαφορετικά false . Εάν πριν την ένθεση ο πίνακας γειτνιάσεως ή ο πίνακας της λίστας γειτνιάσεως δεν έχει διαθέσιμες θέσεις προς πλήρωση αυξάνεται η χωρητικότητα του πίνακα.
<code>bool rmvVtx(const T& info);</code>	Διαγράφει τον κόμβο από τον γράφο, εφόσον ο συγκεκριμένος κόμβος υπάρχει ήδη σε αυτόν. Επιστρέφει true σε περίπτωση επιτυχίας, διαφορετικά false .
<code>bool addEdg(const T& from, const T& to, int cost);</code>	Ενθέτει την ακμή με αφετηρία τον κόμβο <i>from</i> και προορισμό τον κόμβο <i>to</i> και κόστος <i>cost</i> . Απαραίτητες προϋποθέσεις για την επιτυχή ένθεση της ακμής είναι τα εξής: - οι κόμβοι <i>from</i> και <i>to</i> θα πρέπει να υπάρχουν στο γράφημα. - η ακμή με αφετηρία τον κόμβο <i>from</i> και προορισμό τον κόμβο <i>to</i> δεν πρέπει να υπάρχει στο γράφημα. Σημείωση: Σε ένα μη κατευθυνόμενο γράφημα η σειρά δήλωσης των κόμβων <i>from</i> και <i>to</i> από τους οποίους προσδιορίζεται η ακμή δεν έχει σημασία.
<code>bool rmvEdg(const T& from, const T& to);</code>	Διαγράφει την ακμή με αφετηρία τον κόμβο <i>from</i> και προορισμό τον κόμβο <i>to</i> και κόστος <i>cost</i> . Για την επιτυχή διαγραφή της ακμής, αυτή θα πρέπει να υπάρχει στο γράφημα. Σημείωση: Σε ένα μη κατευθυνόμενο γράφημα η σειρά δήλωσης των κόμβων <i>from</i> και <i>to</i> από τους οποίους προσδιορίζεται η ακμή δεν έχει σημασία.
<code>std::list<T> dfs(const T&</code>	Επιστρέφει μία λίστα των κόμβων του γραφήματος, διατρέχοντας το γράφημα

<pre>info) const;</pre>	κατά βάθος (depth first search traversal), με αφετηρία τον κόμβο info. Ο αλγόριθμος εφαρμόζεται σε κατευθυνόμενα και μη κατευθυνόμενα γραφήματα. Η σειρά επίσκεψης των γειτονικών κόμβων ενός κόμβου είναι με βάση τη σειρά εισαγωγής τους στο γράφημα (αυτός που εισήχθη πρώτος, εκτυπώνεται πρώτος, αυτός που εισήχθη δεύτερος εκτυπώνεται δεύτερος κ.ο.κ.).
<pre>std::list<T> bfs(const T& info) const;</pre>	Επιστρέφει μία λίστα των κόμβων του γραφήματος, διατρέχοντας το γράφημα κατά πλάτος (breadth first search traversal), με αφετηρία τον κόμβο info. Ο αλγόριθμος εφαρμόζεται σε κατευθυνόμενα και μη κατευθυνόμενα γραφήματα. Η σειρά επίσκεψης των γειτονικών κόμβων ενός κόμβου είναι με βάση τη σειρά εισαγωγής τους στο γράφημα (αυτός που εισήχθη πρώτος, εκτυπώνεται πρώτος, αυτός που εισήχθη δεύτερος εκτυπώνεται δεύτερος κ.ο.κ.).
<pre>std::list<Edge<T>> mst();</pre>	Υλοποιεί το ελάχιστο επικαλυπτόμενο δέντρο για ένα <u>μη κατευθυνόμενο και συνεκτικό</u> γράφημα. Εάν το γράφημα είναι <u>κατευθυνόμενο</u> επιστρέφει μία άδεια λίστα. Η λίστα των ακμών θα πρέπει να έχει τα εξής χαρακτηριστικά: - Η σειρά των κόμβων κάθε ακμής είναι <u>πρώτα ο κόμβος που προστέθηκε πρώτος στο γράφημα και μετά ο έτερος κόμβος</u>. Ακολουθεί το κόστος της ακμής. - Η σειρά των ακμών είναι από την ακμή χαμηλότερου κόστους προς την ακμή υψηλότερου κόστους. Μεταξύ δύο ακμών ιδίου κόστους δεν υπάρχει συγκεκριμένη προτεραιότητα.
<pre>list<T> dijkstra(const T& from, const T& to);</pre>	Εφαρμόζεται ο αλγόριθμος dijkstra για τη εύρεση του συντομότερου μονοπατιού με αφετηρία τον κόμβο from και προορισμό τον κόμβο to. Επιστρέφεται μία λίστα με τους κόμβους του μονοπατιού από from έως και to. Εάν δεν υπάρχει μονοπάτι μεταξύ from και to επιστρέφεται μία άδεια λίστα.
<pre>list<T> bellman_ford(const T& from, const T& to);</pre>	Εφαρμόζεται ο αλγόριθμος bellman-ford για τη εύρεση του συντομότερου μονοπατιού με αφετηρία τον κόμβο from και προορισμό τον κόμβο to. Επιστρέφεται μία λίστα με τους κόμβους του μονοπατιού από from έως και to. Εάν δεν υπάρχει μονοπάτι μεταξύ from και to επιστρέφεται μία άδεια λίστα. Επιπλέον, ο αλγόριθμος θα πρέπει να μπορεί να ανιχνεύσει εάν υπάρχει βρόγχος αρνητικού βάρους εντός τους γράφου. Σε αυτή την περίπτωση παράγει μία εξαίρεση του τύπου NegativeGraphCycle που είναι απόγονος της κλάσης std::exception. Η μέθοδος what() της παραπάνω εξαίρεσης επιστρέφει το αλφαριθμητικό "Negative Graph Cycle!".
<pre>bool print2DotFile(const char *filename) const;</pre>	Εκτυπώνει το γράφημα σε ένα αρχείο κειμένου κατάλληλο για ανάγνωση και μετασχηματισμό σε εικόνα από το πρόγραμμα graphviz/dot. Επιστρέφει true εάν η εγγραφή ήταν επιτυχής διαφορετικά false. Εάν το αρχείο έχει περιεχόμενο κατά το άνοιγμα, το υφιστάμενο περιεχόμενο διαγράφεται. Η συνάρτηση δεν θα ελεγχθεί από autolab και συνιστάται η χρήση της μόνο για δική σας αποσφαλμάτωση του κώδικα και την επιβεβαίωση της ορθής κατασκευής του γράφου.

Το κυρίως πρόγραμμα

Σας δίνεται ο σκελετός της παραμετρικής συνάρτησης **graphUI** η οποία αποτελεί της υλοποίηση του κυρίως προγράμματος. Η συνάρτηση διαβάζει πάντα από το **stdin** και εκτυπώνει στο **stdout**. Το πρόγραμμα υποθέτει ότι κάθε εντολή καταλαμβάνει πάντα μία ακριβώς γραμμή. Κάθε γραμμή που διαβάζεται από το **stdin** αποθηκεύεται σε ένα **std::stringstream**. Στη συνέχεια το **std::stringstream** χρησιμοποιείται για να κατασκευάσει τα αντικείμενα που αντιπροσωπεύουν το περιεχόμενο των κόμβων του γράφου. Η κλάση των αντικειμένων αυτών αποτελεί την παράμετρο της συνάρτησης και την παράμετρο της κλάσης του γράφου.

Κάθε κλάση που δημιουργεί αντικείμενα τα οποία αποτελούν την πληροφορία των κόμβων του γράφου, θα πρέπει να έχει ένα κατασκευαστή που διαβάζει από **std::istream**, ώστε να μπορεί να κατασκευάσει αντικείμενα διαβάζοντας από το παραπάνω **std::stringstream**. Οι κλάσεις **String**, **Integer** και **Student** που θα χρησιμοποιηθούν στα tests σας δίνονται έτοιμες.

Το κυρίως πρόγραμμα έχει τις εξής εντολές:

Εντολή	Περιγραφή	Εκτύπωση επιτυχίας	Εκτύπωση αποτυχίας
av node	Εντολή εισαγωγής ενός νέου κόμβου με πληροφορία node στο γράφημα. Καλεί τη συνάρτηση addVtx .	av node OK	av node NOK
rv node	Εντολή διαγραφής ενός υφιστάμενου κόμβου με πληροφορία node από το γράφημα. Καλεί τη συνάρτηση rmvVtx .	rv node OK	rv node NOK
ae from to	Εντολή εισαγωγής μιας ακμής στο γράφημα μεταξύ των κόμβων from και to . Καλεί τη συνάρτηση addEdg .	ae from to OK	ae from to NOK
re from to	Εντολή διαγραφής της υφιστάμενης ακμής μεταξύ των κόμβων from και to από το γράφημα. Καλεί τη συνάρτηση rmvEdg .	re from to OK	re from to NOK
dot file	Εντολή εκτύπωσης του γραφήματος σε μορφή dot στο αρχείο με pathname file .	dot file OK	dot file NOK
dfs node	Με αφετηρία τον κόμβο node εκτυπώνει στο stdout η κατά βάθος διαπέραση του γραφήματος. Οι κόμβοι εκτυπώνονται με τη σειρά που επιστρέφονται από τη συνάρτηση dfs . - Αρχικά εκτυπώνεται το αλφαριθμητικό "\n----- DFS Traversal -----\n" . - Στη συνέχεια εκτυπώνεται η διαπέραση του γράφου με αφετηρία τον κόμβο node . Κάθε κόμβος του γράφου διαχωρίζεται από τον προηγούμενο με το αλφαριθμητικό " -> " . - Στο τέλος εκτυπώνεται το αλφαριθμητικό "\n-----\n" .		
bfs node	Με αφετηρία τον κόμβο node εκτυπώνει στο stdout η κατά πλάτος διαπέραση του γραφήματος. Οι κόμβοι εκτυπώνονται με τη σειρά που επιστρέφονται από τη συνάρτηση bfs . - Αρχικά εκτυπώνεται το αλφαριθμητικό "\n----- BFS Traversal -----\n" . - Στη συνέχεια εκτυπώνεται η διαπέραση του γράφου με αφετηρία τον κόμβο node . Κάθε κόμβος του γράφου διαχωρίζεται από τον προηγούμενο με το αλφαριθμητικό " -> " . - Στο τέλος εκτυπώνεται το αλφαριθμητικό "\n-----\n" .		
dijkstra from to	Εκτυπώνεται το αλφαριθμητικό "Dijkstra: " ακολουθούμενο από το μονοπάτι των κόμβων από τον κόμβο from έως τον κόμβο to . Κάθε κόμβος διαχωρίζεται από τον προηγούμενο με το αλφαριθμητικό " , " (κόμμα και κενό).		
bellman-ford from to	Εκτυπώνεται το αλφαριθμητικό "Bellman-Ford: " . Εάν δεν υπάρχει βρόγχος αρνητικού βάρους μέσα στον γράφο εκτυπώνεται το μονοπάτι των κόμβων από τον κόμβο from έως τον κόμβο to . Κάθε κόμβος διαχωρίζεται από τον προηγούμενο με το αλφαριθμητικό " , " (κόμμα και κενό). Στην περίπτωση που υπάρχει βρόγχος αρνητικού βάρους, θα πρέπει να εκτυπώνεται το μήνυμα "Negative Graph Cycle!" ακολουθούμενο από χαρακτήρα αλλαγής γραμμής.		
mst	Εκτυπώνεται το αλφαριθμητικό "\n--- Min Spanning Tree ---\n" . Στη συνέχεια εκτυπώνονται οι		

	ακμές του ελάχιστου επικαλυπτόμενου δέντρου, με τη σειρά που επιστρέφονται από τη συνάρτηση mst . Τέλος, εκτυπώνεται το συνολικό κόστος διαπέρασης του δέντρου που αποτελεί το άθροισμα του κόστους των επιμέρους ακμών. Στο τέλος εκτυπώνεται το αλφαριθμητικό " MST Cost: x " ακολουθούμενο από χαρακτήρα αλλαγής γραμμής όπου x το συνολικό κόστος του δέντρου.
#	Μία εντολή που ξεκινά με τον χαρακτήρα <code>`#'</code> και ακολουθεί κενός χαρακτήρας δε λαμβάνεται υπόψη από το πρόγραμμα.
α	Μία εντολή που ξεκινά με τον χαρακτήρα <code>`α'</code> και ακολουθεί κενός χαρακτήρας σηματοδοτεί τον τερματισμό του προγράμματος.

Έλεγχος του προγράμματος σας

Για τον έλεγχο του προγράμματος σας παρέχονται τα παρακάτω tests:

`bfs1`, `bfs2`, `bfs3`, `dfs1`, `dfs2`, `dfs3`, `mst1`, `mst2`, `mst3`, `dijkstra1`, `dijkstra2`, `dijkstra3`, `dijkstra4`, `bellman-ford1`, `bellman-ford2`, `bellman-ford3`.

[Τις εικόνες από τα γραφήματα που αντιστοιχούν στα αρχεία ελέγχου μπορείτε να τις κατεβάσετε εδώ.](#)

Makefile και αρχεία ελέγχου

Σας παρέχεται ο αρχείο [skeleton.tar.gz](#) (περιέχει και τα παραπάνω tests) το οποίο διαθέτει τους σκελετούς των αρχείων που σας δίνονται έτοιμα, κατάλληλο Makefile και τα αρχεία ελέγχου (βρίσκονται στον υποκατάλογο **tests**). Για να εκτελέσετε όλα τα test γράφετε στο terminal `$> make run`, ενώ για να εκτελέσετε ένα μεμονωμένο test (π.χ. `dijkstra1`) γράφετε `$> make dijkstra1`.

Μέσα στο αρχείο `skeleton.tar.gz` υπάρχει και το αρχείο `UnionFind.hpp`, το οποίο παρέχεται κενό. Σε περίπτωση που θέλετε να υλοποιήσετε ένωση-εύρεση σε ξένα σύνολα αποθηκεύστε τον κώδικα σας στο αρχείο αυτό. Εάν δεν υλοποιήσετε ένωση-εύρεση αφήστε το κενό, αλλά υποβάλλετε το μαζί με τα άλλα δύο αρχεία (δείτε σχετικά στην ενότητα Τρόπος Αποστολής)

Τρόπος Αποστολής

Η εργασία θα εξεταστεί μέσω του εργαλείου αυτόματης υποβολής και διόρθωσης autolab. Οι οδηγίες αποστολής είναι οι εξής:

1. Συμπίεστε τα αρχεία **Graph.hpp**, **GraphUI.hpp** και **UnionFind.hpp** σε ένα αρχείο ZIP με όνομα **hw4.zip**. Το αρχείο **UnionFind.hpp** σας δίνεται κενό. Εάν θέλετε να υλοποιήσετε μία κλάση ένωσης-εύρεσης σε ξένα σύνολα μπορείτε να το κάνετε στο αρχείο αυτό. Σε κάθε περίπτωση το αρχείο πρέπει να υποβληθεί (ακόμη και κενό περιεχομένου).
2. Συνδέεστε στο autolab και επιλέγετε το μάθημα **ECE326_2024 (S22)** και από αυτό την εργασία **HW4**.
3. Για να υποβάλλετε την εργασία σας κάνετε click στην επιλογή "I affirm that I have compiled with this course academic integrity policy..." και πατάτε submit. Στη συνέχεια επιλέγετε το αρχείο **hw4.zip** που δημιουργήσατε παραπάνω.

Παράρτημα 1 - Ενδεικτικά γραφήματα

Τα παρακάτω γραφήματα είναι ενδεικτικά για τις δοκιμές σας. Σε καμία περίπτωση δεν αποτελούν τα μοναδικά γραφήματα τα οποία θα εξεταστούν στα test cases.



